TASKQUEST: STUDENT TASK TRACKER

Project Report

PREPARED BY: Rohith V

1. Executive Summary & Abstract

TaskQuest is a production-grade, secure academic management application designed to help students organize tasks, manage courses, track deadlines, and monitor performance. Utilizing a decoupled MVC architecture, the system is built with a lightweight Vanilla HTML5/CSS3/JavaScript front-end and a robust Node.js/Express.js/MongoDB REST API backend. System security is enforced via cryptographic password hashing using bcrypt, JSON Web Token (JWT) access/refresh token pairs, rate limiting, and input sanitization.

2. System Architecture & MVC Layout

The codebase strictly adheres to the Model-View-Controller pattern:
• Views (Client Layer): Rich interface featuring a responsive design system, custom light/dark theme variables, dynamic glassmorphic panels, and interactive dashboards driven by Chart.js.
• Controllers (API Layer): Orchestrates registration/login cycles, task CRUD, pagination, bulk updates/deletions, and security audits.
• Models (Database Layer): Mongoose collections with compound indexes. Data models consist of Users, Categories (color-coded courses), Tasks (with priority, status, and due dates), and AuditLogs.

3. Core Functional & Non-Functional Requirements

Functional Requirements:

• User Management: Secure login/logout, password changes, and mock password recovery links.
• Task & Category CRUD: Task creation with color-coded category labels. Category deletions cascade-default tasks to 'General'.
• Bulk Operations: Allows users to perform batch-complete or batch-delete actions on multiple selected tasks at once.
• Background Daemon: Scans the database and dispatches notifications for upcoming task deadlines.

Non-Functional Security Controls:

• Cryptographic Hashing: Passwords are encrypted using bcryptjs with a work factor of 10.
• JWT Auth Pairs: Short-lived access tokens (15m) paired with long-lived refresh tokens (7d) stored securely.
• Request Defense: Implements express-rate-limit to mitigate DDoS/brute force and express-mongo-sanitize to intercept NoSQL injection payloads.

4. Database Design & Schema Architecture

TaskQuest structures its documents across 5 main MongoDB collections:
1. Users Collection: Stores usernames, emails, hashed passwords, roles (student/admin), and lists of active refresh tokens.
2. Categories Collection: Custom course identifiers with hex color values and references to their owner user.
3. Tasks Collection: Contains due dates, priorities (low/medium/high), status tags (pending/in-progress/completed), and references to Categories and Users.
4. AuditLogs Collection: Immutable logging stream capturing transaction actions, IP addresses, browser headers, and detailed payload states for administrative security review.
5. Reminders Collection: Maps task due dates to notification triggers evaluated by the scheduler.

5. Testing Methodology & Verification

To ensure continuous integration readiness, the core REST API is verified using Jest and Supertest. Because CI environments are typically database-free, we engineered a custom, complete database simulation layer:
• In-Memory Database Simulator (tests/mongooseMock.js): Intercepts Mongoose ODM, matching collections, schemas, aggregates, populations, queries, and chain builders.
• Isolation: The mock store resets before every test run, preventing data leaks between test files.
• Test Cases: Verification covers Alice signup (preferences setup), login validation, route authorization guards, custom task lists, query filters, and bulk operation processing.
• Results: 9 major integration tests executed with a 100% success rate, zero leaks, and zero open handles.

6. Key Engineering Challenges

• Port Conflicts in Jest: Parallel execution caused EADDRINUSE errors on port 5000. Resolved by disabling server listener calls when NODE_ENV = 'test', enabling Supertest to deploy on random ports.
• Virtual Mongoose ID Matching: Controllers searched for user profiles via virtual getters, which default mocks lacked. Resolved by modifying the mock constructor to map virtual id getters directly to ObjectIds.
• Category Cascade Deletion: Deleting categories threatened to orphan tasks. Resolved by adding a pre-delete middleware hook that automatically updates associated tasks to point to a fallback 'General' category.

7. Conclusion & Future Roadmap

TaskQuest is an operational, highly secure capstone application ready for production cloud environments. Future plans for expansion include:
1. Live Google Calendar Synchronization: Bi-directional synchronization of task deadlines with Google Calendar API.
2. Shared Boards: Enabling multiple students to collaborate on shared task/project categories.
3. Real-Time Web Sockets: Transitioning from polling to socket.io-based instant notification push feeds.

Key References:

• Field, R. 'Clean Architecture: A Craftsman's Guide to Software Structure', Prentice Hall, 2017.
• OWASP Top 10 Security Reference Guide, 2025.